

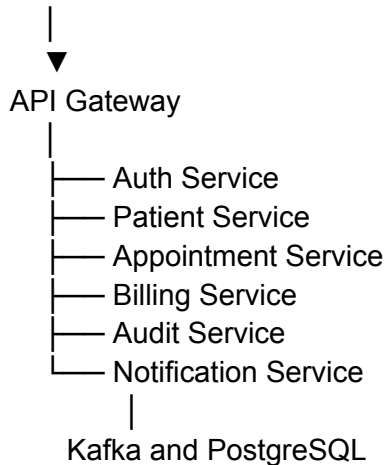
Full HealthFlow Development Plan

The best approach is to turn the current project into a small but complete healthcare administration platform. Build it in layers, understand each layer, and return to Kubernetes only after the application works locally.

A realistic part-time learning timeline is approximately 12–16 weeks. Treat the milestones as completion gates rather than strict deadlines.

Final product

HealthFlow Web Application



The finished application should support:

- Staff login
 - Role-based access
 - Patient management
 - Appointment scheduling
 - Billing and invoices
 - Notifications
 - Audit history
 - Dashboard statistics
 - Local Docker deployment
 - Kubernetes/AWS deployment
-

Milestone 0 — Establish a clean Git starting point

Your local branch currently contains the Phase 12 revert, but GitHub still contains the completed Phase 12 commit.

First publish the rollback:

```
git push origin kubernetes
```

Then create a separate development branch:

```
git switch -c healthflow-frontend  
git push -u origin healthflow-frontend
```

Use smaller branches for individual features:

```
feature/frontend-foundation  
feature/patient-search  
feature/appointments  
feature/billing-persistence  
feature/notifications  
feature/role-security
```

Completion criteria:

- Local and remote branches are synchronized.
- Phase 12 remains available in Git history.
- New development happens outside the `kubernetes` branch.

Milestone 1 — Define the application requirements

Before writing code, define the users.

Roles

Administrator

- Manage staff users
- View patients
- View appointments
- View billing
- View audit logs
- View system dashboard

Receptionist

- Register patients
- Update patients
- Schedule appointments
- Cancel or reschedule appointments

Doctor

- View assigned appointments
- View patient information
- Complete appointments
- Add clinical notes

Billing staff

- View patient billing accounts
- Create invoices
- Record payments
- View unpaid invoices

Do not introduce all roles immediately. Begin with **ADMIN**, then add other roles after the basic frontend works.

Main workflows

Write these down before implementation:

1. Staff member logs in.
2. Staff member registers a patient.
3. Billing account is created automatically.
4. Audit and notification events are produced.
5. Receptionist schedules an appointment.
6. Doctor completes the appointment.
7. Billing staff creates or reviews an invoice.
8. Administrator reviews the audit history.

Completion criteria:

- Roles are documented.
 - Pages are listed.
 - Each page has an identified backend endpoint.
 - You understand the main user workflow.
-

Milestone 2 — Stabilize the current backend

Do this before building the frontend.

2.1 Database migrations

Return to Flyway or another migration system so database schemas are managed by each service.

Recommended ownership:

Auth Service → auth_db
Patient Service → patient_db
Appointment → appointment_db
Billing Service → billing_db
Audit Service → audit_db
Notification → notification_db

Initially, these can still run inside one PostgreSQL container. Separate logical databases are sufficient for learning.

2.2 Standard API errors

Every service should return a consistent error structure:

Timestamp
HTTP status
Error code
Message
Request path
Field validation errors

The frontend can then display useful messages instead of generic failures.

2.3 Health endpoints

Every service should expose:

/actuator/health

/actuator/info

This will later be used by Docker and Kubernetes health checks.

2.4 Gateway CORS configuration

The browser frontend will probably run on a different port during development. Configure CORS centrally in the API Gateway.

Do not depend only on `@CrossOrigin` inside Patient Service.

2.5 API documentation

Generate OpenAPI documentation for each public endpoint. Record:

- Request body
- Response body
- Validation rules
- Required role
- Possible error responses

Completion criteria:

- Every service starts successfully.
- Database creation is repeatable.
- Health endpoints work.
- Browser requests can pass through the Gateway.
- API errors follow one format.

Milestone 3 — Improve authentication

The current login and JWT flow is a good foundation, but the frontend needs more support.

Required endpoints

POST /auth/login
POST /auth/validate
GET /auth/me

GET /auth/me should return information such as:

User ID
Email
Role

Later endpoints

POST /auth/users
GET /auth/users
GET /auth/users/{id}
PATCH /auth/users/{id}/role
PATCH /auth/users/{id}/status

Frontend authentication behavior

- User logs in.
- JWT is stored for the session.
- HTTP requests include `Authorization: Bearer <token>`.
- Invalid or expired tokens redirect to login.
- Protected routes cannot be opened without authentication.
- Navigation options depend on the user's role.

For this learning project, start with a short-lived access token. Add refresh tokens only after the simpler flow works.

Completion criteria:

- Login works through the API Gateway.
- /auth/me returns the current user.
- Invalid tokens produce `401 Unauthorized`.
- Roles are enforced by the backend, not just hidden in the frontend.

Milestone 4 — Improve Patient Service

The current Patient Service already supports CRUD. Improve it before building a sophisticated patient screen.

Add pagination

GET /api/patients?page=0&size=20

Return:

Patients

Current page

Page size

Total elements

Total pages

Add searching and filtering

GET /api/patients?search=John

GET /api/patients?active=true

GET /api/patients?search=John&page=0&size=20

Search by:

- Name
- Email
- Patient ID

Use soft deletion

Healthcare-style records should not disappear from normal history. Instead of deleting the database row, mark the patient inactive.

Possible endpoint:

PATCH /api/patients/{id}/status

Add dashboard statistics

GET /api/patients/statistics

Possible response:

- Total patients
- Active patients
- Inactive patients

- Registrations this month

Completion criteria:

- Patient list supports pagination.
 - Search works.
 - Active/inactive filtering works.
 - Patient data is validated.
 - Dashboard statistics are available.
-

Milestone 5 — Build the first frontend version

Recommended starting stack

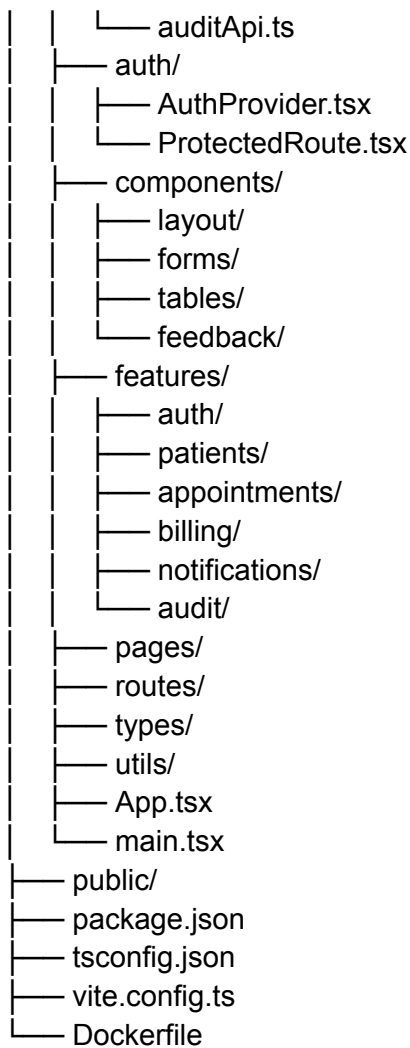
For a learning-focused single-page application:

- React
- TypeScript
- Vite
- React Router
- TanStack Query
- A component library or your own CSS system
- A form library with schema validation

Create React App is deprecated, and React's documentation lists Vite as an appropriate build tool for a from-scratch React application. [React installation guidance](#), [React build-from-scratch guidance](#). TanStack Query is useful for managing loading, caching, mutation and refetching of server data. [TanStack Query documentation](#).

Suggested directory structure

```
frontend/
├── src/
│   ├── api/
│   │   ├── httpClient.ts
│   │   ├── authApi.ts
│   │   ├── patientApi.ts
│   │   ├── appointmentApi.ts
│   │   └── billingApi.ts
```



Organize most application code by feature. Avoid placing every component in one large shared folder.

First frontend pages

Login

- Email and password fields
- Form validation
- Loading indicator
- Invalid credentials message
- Redirect after successful login

Application layout

- Sidebar

- Top navigation
- User menu
- Logout button
- Mobile navigation

Patient list

- Table
- Search box
- Active/inactive filter
- Pagination
- Create-patient button
- Edit and view actions

Patient form

- Name
- Email
- Address
- Date of birth
- Client-side validation
- Server error display

Patient details

- Patient information
- Status
- Registration date
- Edit action
- Future appointment section
- Future billing section

Audit page

- Event type
- Actor ID
- Timestamp
- Expandable payload
- Filtering

Dashboard

- Total patients
- Active patients

- Recent registrations
- Recent audit events
- Service status

Completion criteria:

- User can log in.
 - Protected routes work.
 - Patient CRUD works through the Gateway.
 - Loading, empty and error states exist.
 - The layout works on desktop and mobile.
-

Milestone 6 — Appointment Service

This is the most valuable new business feature.

Appointment entity

Suggested fields:

id
patientId
doctorId
appointmentDateTime
durationMinutes
reason
status
notes
createdAt
updatedAt

Statuses:

SCHEDULED
CONFIRMED
COMPLETED
CANCELLED
NO_SHOW

Endpoints

POST /api/appointments
GET /api/appointments
GET /api/appointments/{id}
PUT /api/appointments/{id}
PATCH /api/appointments/{id}/status
DELETE /api/appointments/{id}

Filtering:

GET /api/appointments?date=2026-07-20
GET /api/appointments?patientId=10
GET /api/appointments?doctorId=5
GET /api/appointments?status=SCHEDULED

Events

Publish Kafka events:

AppointmentScheduled
AppointmentRescheduled
AppointmentCancelled
AppointmentCompleted

Audit and Notification Services can consume these events.

Frontend pages

- Appointment calendar
- Appointment list
- Create appointment
- Appointment details
- Reschedule dialog
- Cancel confirmation
- Doctor's daily schedule

Completion criteria:

- Appointments can be scheduled and updated.
 - Double booking is prevented.
 - Appointment events appear in audit history.
 - Frontend calendar and list views work.
-

Milestone 7 — Complete Billing Service

The existing Billing Service does not persist billing accounts.

Domain model

Billing account

id
patientId
status
createdAt

Invoice

id
billingAccountId
appointmentId
invoiceNumber
totalAmount
status
dueDate
createdAt

Invoice item

description
quantity
unitPrice
subtotal

Payment

id
invoiceId
amount
paymentMethod
paidAt
reference

Invoice statuses

DRAFT
ISSUED
PARTIALLY_PAID

PAID
OVERDUE
CANCELLED

Endpoints

GET /api/billing/accounts/patient/{patientId}
POST /api/billing/invoices
GET /api/billing/invoices
GET /api/billing/invoices/{id}
POST /api/billing/invoices/{id}/payments

Frontend pages

- Patient billing overview
- Invoice list
- Invoice details
- Create invoice
- Record payment
- Outstanding balances dashboard

Completion criteria:

- Billing accounts are stored.
- Invoices and payments are stored.
- Patient details display billing information.
- Payment status is calculated correctly.
- Billing events create audit records.

Milestone 8 — Complete Notification Service

Store notifications

Suggested fields:

id
recipientUserId
patientId

type
title
message
read
createdAt

Notification types

PATIENT_CREATED
APPOINTMENT_REMINDER
APPOINTMENT_CANCELLED
INVOICE_CREATED
PAYMENT_RECEIVED

Endpoints

GET /api/notifications
GET /api/notifications/unread-count
PATCH /api/notifications/{id}/read
PATCH /api/notifications/read-all

Frontend features

- Notification bell
- Unread badge
- Notification panel
- Notification history
- Mark one or all as read

Initially, notifications can remain in-app notifications. Actual email delivery can be added later.

Completion criteria:

- Kafka events create stored notifications.
- The frontend displays unread notifications.
- Read status persists.
- Notification failures do not break patient or appointment operations.

Milestone 9 — User and role management

Backend security

Apply method-level authorization:

ADMIN
RECEPTIONIST
DOCTOR
BILLING_STAFF

Examples:

- Only administrators manage users.
- Receptionists create patients and appointments.
- Doctors complete appointments.
- Billing staff manage invoices and payments.
- Audit logs are restricted.

Frontend pages

- User list
- Create staff user
- Change role
- Activate/deactivate user

The frontend should hide inaccessible features, but the backend must still reject unauthorized requests.

Completion criteria:

- Each endpoint has defined role permissions.
- Unauthorized requests return **403 Forbidden**.
- Navigation changes according to role.
- Disabling a user prevents future access.

Milestone 10 — Testing strategy

Backend unit tests

Test:

- Service business rules
- Duplicate email handling
- Authentication
- Appointment conflicts
- Invoice calculations
- Role authorization

Backend integration tests

Test:

- Repository queries
- PostgreSQL behavior
- Kafka publishing and consuming
- Gateway routing
- Full login-to-patient workflow

Frontend tests

Test:

- Form validation
- Login behavior
- Protected routes
- Patient table
- Error messages
- Role-based navigation

End-to-end tests

Important workflows:

1. Log in.
2. Create a patient.
3. Confirm billing account creation.
4. Confirm audit entry.
5. Schedule appointment.
6. Complete appointment.
7. Create invoice.
8. Record payment.
9. Confirm notification.

Completion criteria:

- Critical business rules have unit tests.
 - Main endpoints have integration tests.
 - At least one complete browser workflow is automated.
 - Tests run from a single documented command.
-

Milestone 11 — Local Docker environment

Update Docker Compose to run:

- Frontend
- API Gateway
- All microservices
- PostgreSQL
- Kafka
- ZooKeeper or the selected Kafka mode

Only expose necessary ports to the host:

Frontend → localhost:3000 or localhost:80

API Gateway → localhost:8084

Internal services should communicate using Docker service names.

Add:

- Health checks
- Persistent database volumes
- Development environment examples
- Startup documentation
- Reset instructions

Do not commit:

- Real passwords
- JWT secrets
- `.env`
- Database data
- Terraform provider caches

Completion criteria:

- One command starts the complete local application.

- The frontend communicates only through the Gateway.
 - Containers become healthy.
 - Data survives container restarts.
-

Milestone 12 — Observability and reliability

Add:

- Structured logs
- Request/correlation IDs
- Service health dashboard
- Basic metrics
- Kafka consumer error handling
- Retry behavior
- Dead-letter topics
- Database transaction boundaries

For the patient-creation workflow, decide what should happen when:

- Patient saves successfully but Billing Service fails.
- Patient saves successfully but Kafka is unavailable.
- Audit processing fails.
- Notification processing fails.

This is where you learn distributed-system reliability.

Completion criteria:

- One request can be traced across services.
- Failed Kafka messages are not silently lost.
- Service failures produce understandable logs.
- The system reports degraded dependencies.